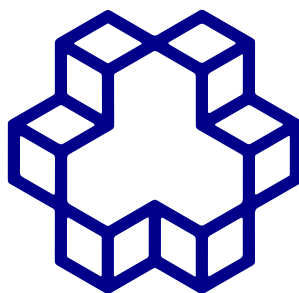


به نام خدا



دانشگاه صنعتی خواجه نصیرالدین طوسی

گزارش پروژه پایانی درس یادگیری تقویتی

طراحی و تحلیل عامل هوشمند در هزارتوی پویا

نام و نام خانوادگی: ابراهیم فردمنفرد

شماره دانشجویی: ۴۰۴۱۱۵۰۴

مخزن **GitHub**: [RL_FinalProject_40411504](#)

۱. مقدمه

این گزارش، مستندسازی کامل پروژه پایانی درس یادگیری تقویتی است. هدف این پروژه، طراحی یک محیط هزارتوی پویا به صورت فرایند تصمیم‌گیری مارکوف (MDP)، پیاده‌سازی سه الگوریتم Value Iteration، Q-Learning و $SARSA(\lambda)$ از پایه (بدون استفاده از کتابخانه‌های آماده RL)، مقایسه عملکرد آن‌ها، اجرای یک بخش محدود انتقال یادگیری روی الگوریتم Q-Learning، و در نهایت ارائه نتایج در قالب رابط گرافیکی، نمودارهای آماری و تحلیل کمی بوده است.

تمامی کدها، داده‌های خام، مدل‌های ذخیره‌شده و تصاویر این گزارش مستقیماً از مخزن پروژه قابل بازتولید هستند. دستور اجرای هر بخش در فایل 'README.md' مخزن قرار داده شده است.

۲. تعریف مسئله به صورت MDP

۲.۱. محیط

محیط شبیه‌سازی‌شده، یک هزارتوی دوبعدی 15×15 است. بر اساس فرمول مندرج در سند پروژه و با توجه به رقم یکی مانده به آخر شماره دانشجویی (۰)، ابعاد و Seed پایه به شکل زیر محاسبه شده است:

$$b = 0, \quad N = 15 + (0 \bmod 4) = 15$$

این ابعاد با پیاده‌سازی انجام‌شده ($grid_size = 15$) و مقدار Seed پایه ($seed = 0$) کاملاً هم‌خوانی دارد. نقشه توسط تابع `_build_grid()` در فایل `environments/maze.py` به صورت تصادفی (با Seed ثابت و تکرارپذیر) ساخته می‌شود. این نقشه شامل ۳۵ خانه دیوار (معادل ۱۵.۶٪ از کل ۲۲۵ خانه، که بالاتر از حد نصاب ۱۵٪ است) و ۸ خانه جریمه (بالاتر از حد نصاب ۵ خانه) است.

* نقطه شروع: (0,0)

* کلید: (2,12)

* در قفل شده: (13,14)

* هدف: (14,14)

پس از هر بار ساخت نقشه، تابع `_bfs_check()` با استفاده از الگوریتم جست و جوی سطح اول (BFS)، وجود مسیر معتبر (شروع به کلید و کلید به هدف) را تأیید می کند. در صورت نامعتبر بودن نقشه، تابع `_generate_valid_map()` تا سقف ۱۰۰۰ تلاش، نقشه را دوباره می سازد. این مکانیزم دقیقاً پیاده سازی الزام «اعتبارسنجی قطعی با BFS و بازتولید تکرارپذیر» است.

۲.۲. فضای حالت و حفظ خاصیت مارکوف

قابلیت تکمیلی انتخاب شده برای این پروژه **مانع متحرک (Moving Obstacle)** است. این مانع در یک مسیر حلقه ای هشت خانه ای زیر حرکت می کند :

`patrol_route = [(7,4), (7,5), (7,6), (8,6), (9,6), (9,5), (9,4), (8,4)]`

در هر گام، مانع یک خانه در این مسیر جابه جا می شود و برخورد عامل با موقعیت فعلی مانع، جریمه ی ۱۵- به همراه دارد. از آنجا که موقعیت مانع در گام بعد تنها به موقعیت فعلی اش وابسته است (نه به تاریخچه)، برای حفظ خاصیت مارکوف کافی است اندیس مانع در مسیر حلقه ای (p) نیز بخشی از حالت باشد. بنابراین حالت به صورت زیر تعریف شده است:

$$s = (x, y, k, p) \quad k \in \{0,1\}, \quad p \in \{0, \dots, 7\}$$

این فرمول در تابع `get_state()` پیاده‌سازی شده است. با دانستن حالت S و عمل انتخابی a ، توزیع احتمال حالت بعدی (S') کاملاً مشخص است و نیازی به تاریخچه‌ی مراحل قبلی نیست؛ که این موضوع مفهوم دقیق خاصیت مارکوف را نشان می‌دهد. تعداد کل حالت‌های معتبر محیط برابر است با:

$$(225 - 35 \text{ دیوار}) \times 2 \times 8 = 3040 \text{ حالت.}$$

۲.۳. فضای عمل و تابع انتقال

فضای عمل گسسته و شامل چهار عمل {بالا، 1: راست، 2: پایین، 3: چپ، 0: است. طبق تابع

`step()`:

- با احتمال ۰,۸ عمل انتخابی همان‌طور اجرا می‌شود.
- و با احتمال ۰,۱ به جهت عمود دیگر $((a-1) \bmod 4)$ با احتمال ۰,۱ عامل به یک جهت عمود منحرف می‌شود $((a+1) \bmod 4)$.
- برخورد با دیوار یا مرز نقشه یا تلاش برای عبور از در بسته‌بدون کلید، عامل را در همان خانه نگه می‌دارد.

این عدم قطعیت هم در تابع `step()` (برای روش‌های Q-Learning و SARSA) و هم در تابع

`get_transitions()` به‌عنوان مدل کامل احتمالاتی برای Value Iteration (به‌صورت یکسان

پیاده‌سازی شده است).

۲.۴. تابع پاداش — دو نسخه

نسخه‌ی Sparse (پیش فرض) :

استدلال	پاداش	رویداد
هزینه‌ی کوچک برای تشویق عامل به یافتن مسیر کوتاه	-۱	هر حرکت
جریمه‌ی خفیف اضافه بر هزینه‌ی حرکت، بدون غلبه بر سیگنال اصلی	-۵	برخورد با دیوار/در بسته
باز خورد قابل توجه برای اجتناب از نواحی خطرناک	-۱۰	ورود به خانه جریمه
جریمه‌ای بیشتر از خانه‌ی جریمه‌ی ثابت، تا قابلیت تکمیلی تأثیر واقعی داشته باشد	-۱۵	برخورد با مانع متحرک
پاداش میان‌راهی بزرگ که پیشرفت واقعی را نشان می‌دهد	+۵۰	دریافت کلید
بزرگ‌ترین پاداش، زیرا هدف نهایی است	+۱۰۰	رسیدن به هدف

نسخه‌ی Shaped (شکل‌دهی‌شده): این نسخه بر پایه‌ی فرم استاندارد

Potential-Based Reward Shaping توسعه یافته است. تابع پتانسیل مبتنی بر فاصله

منهتن منفی تا هدف فعلی است (در صورتی که کلید گرفته نشده باشد فاصله تا کلید، و در غیر این صورت فاصله تا هدف نهایی محاسبه می‌شود):

$$\Phi(s) = -(|x - x_{target}| + |y - y_{target}|), \quad F(s, a, s') = \gamma \Phi(s') - \Phi(s)$$

$$R_{shaped} = R_{sparse} + F(s, a, s')$$

از آنجا که این فرمول Potential-Based است، بر اساس قضایای اثبات‌شده در ادبیات RL، سیاست بهینه‌ی MDP را تغییر نمی‌دهد اما سرعت یادگیری را به شدت افزایش می‌دهد. تأثیر دقیق این موضوع در بخش ۱۰ بررسی شده است.

۲.۵. ضریب کاهش، حالت‌های پایانی و سیاست

ضریب کاهش (تنزیل) پیش‌فرض $\gamma = 0.9$ است. اثر مقادیر دیگر در بخش Value Iteration ارزیابی شده است. تنها حالت پایانی محیط، رسیدن عامل به خانه‌ی هدف (14,14) به شرط داشتن کلید ($k = 1$) است که وضعیت را به `done=True` تغییر می‌دهد. سیاست عامل $\pi(s)$ در هر سه الگوریتم به‌صورت حریصانه (Greedy) نسبت به تابع ارزش نهایی استخراج می‌شود.

۲.۶. سقف اپیزود

سقف تعداد گام‌ها در هر اپیزود ۵۰۰ تعیین شده است (`max_steps=500` در فایل `settings.json`). این مقدار تقریباً برابر با سه برابر تعداد خانه‌های قابل عبور (۱۹۰ خانه غیردیوار) است. عبور از این سقف بدون رسیدن به هدف، اپیزود را خاتمه داده اما وضعیت به `done=False` باقی می‌ماند.

۳. نمای کلی پیاده‌سازی

مؤلفه	فایل
محیط و مدل انتقال	environments/maze.py
BFS + تولید نقشه مستقل	environments/generator.py
Value Iteration	agents/value_iteration.py
Q-Learning	agents/q_learning.py
SARSA(λ)	agents/sarsa_lambda.py
انتقال یادگیری	transfer/transfer_learning.py
رابط گرافیکی	gui/app.py, gui/renderer.py
اجرای آزمایش‌ها و تحلیل	main.py, experiments/run_experiments.py, experiments/analysis.py

۴. الگوریتم اول : Value Iteration

الگوریتم Value Iteration به صورت کاملاً مستقل و بدون استفاده از کتابخانه‌های آماده پیاده‌سازی شده است. برخلاف دو الگوریتم دیگر، این روش به مدل کامل انتقال محیط (`env.get_transitions`) یعنی احتمالات $0.1/0.1/0.8$ دسترسی مستقیم دارد. به روزرسانی بلمن:

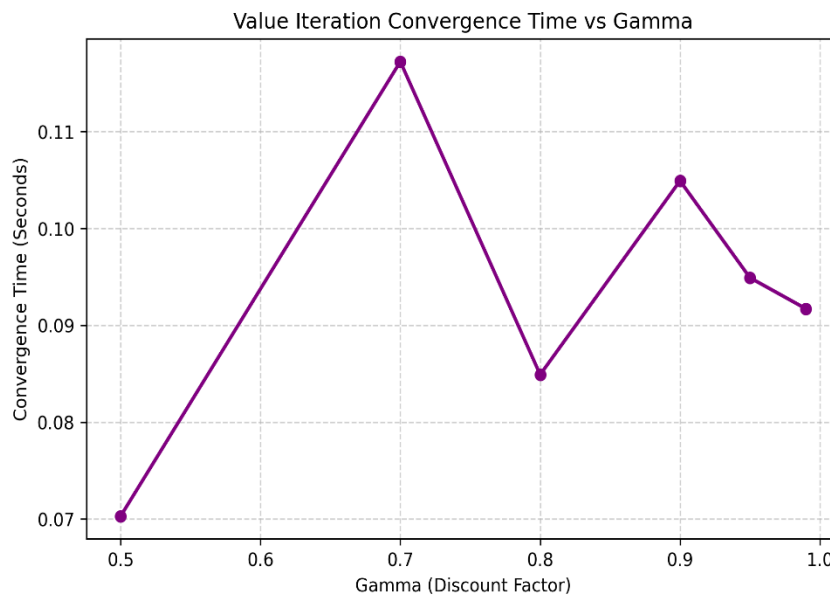
$$V_{k+1}(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V_k(s')]$$

شرط همگرایی زمانی محقق می‌شود که حداکثر تغییر $|V_{k+1}(s) - V_k(s)|$ روی تمامی حالت‌ها کمتر از آستانه‌ی $\theta = 1e - 4$ شود. سیاست نهایی با انتخاب حریصانه‌ی عمل بهینه از مقادیر V^* استخراج می‌گردد.

۴.۱. اثر ضریب تنزیل (γ)

آزمایش بر روی ۶ مقدار مختلف برای γ اجرا و در فایل `vi_convergence_vs_gamma.csv` ذخیره شده است:

γ	تعداد تکرار تا همگرایی	زمان اجرا (ثانیه)
0.50	28	0.070
0.70	46	0.117
0.80	43	0.085
0.90	48	0.105
0.95	45	0.095
0.99	42	0.092



$\gamma = 0.5$

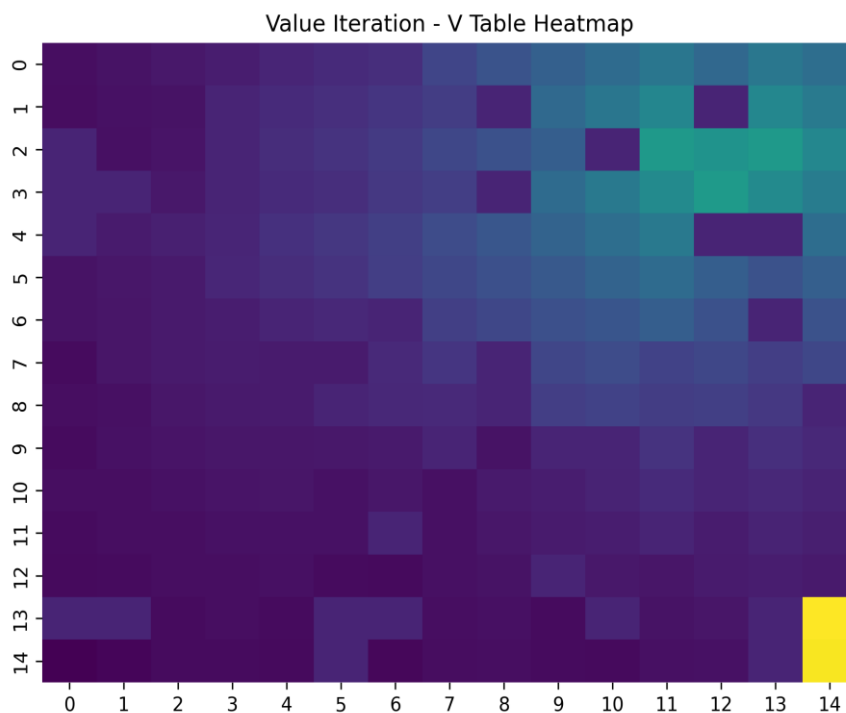
تحلیل: با مقدار

همگرایی بسیار سریع تر (۲۸ تکرار) رخ می دهد، زیرا افق برنامه ریزی کوتاه است و اثر پاداش های

دوردست به سرعت میرا می شود؛ در واقع تابع ارزش عملاً فقط چند گام جلوتر را می بیند. با $\gamma \geq$

0.7 تعداد تکرارها در بازه‌ی ۴۲ تا ۴۸ نسبتاً باثبات می‌ماند. نوسان جزئی (مثلاً ۴۸ تکرار برای $\gamma = 0.9$ در برابر ۴۲ تکرار برای $\gamma = 0.99$) نشان می‌دهد که در این محیط با افق نسبتاً کوچک، افزایش بیشتر γ لزوماً همگرایی را کندتر نمی‌کند. عامل اصلی، تعداد گام‌هایی است که اطلاعات پاداش هدف باید به عقب منتشر شود که وابسته به توپولوژی هزار تو است. زمان اجرای بسیار پایین (کمتر از ۰.۲ ثانیه برای ۳۰۴۰ حالت) نشان می‌دهد که الگوریتم VI بسیار کارآمد پیاده‌سازی شده است.

۴.۲. ارزش و سیاست بهینه Heatmap



تحلیل :ارزش حالت‌ها به‌وضوح با فاصله تا هدف (از طریق مسیر کلید و در) نسبت مستقیم دارد . خانه‌های نزدیک نقطه‌ی شروع در گوشه‌ی بالا-چپ منفی‌ترین ارزش را دارند، زیرا مسیر طولانی و پرخطری تا هدف باقی مانده است . در حالی که خانه‌های نزدیک در و هدف، بیشترین (روشن‌ترین) مقدار را نشان می‌دهند . الگوی ناپیوسته در برخی نواحی، نمایانگر اثر دیوارها و خانه‌های جریمه است که مسیر بهینه را منحرف می‌کنند .

۵. الگوریتم دوم: Q-Learning

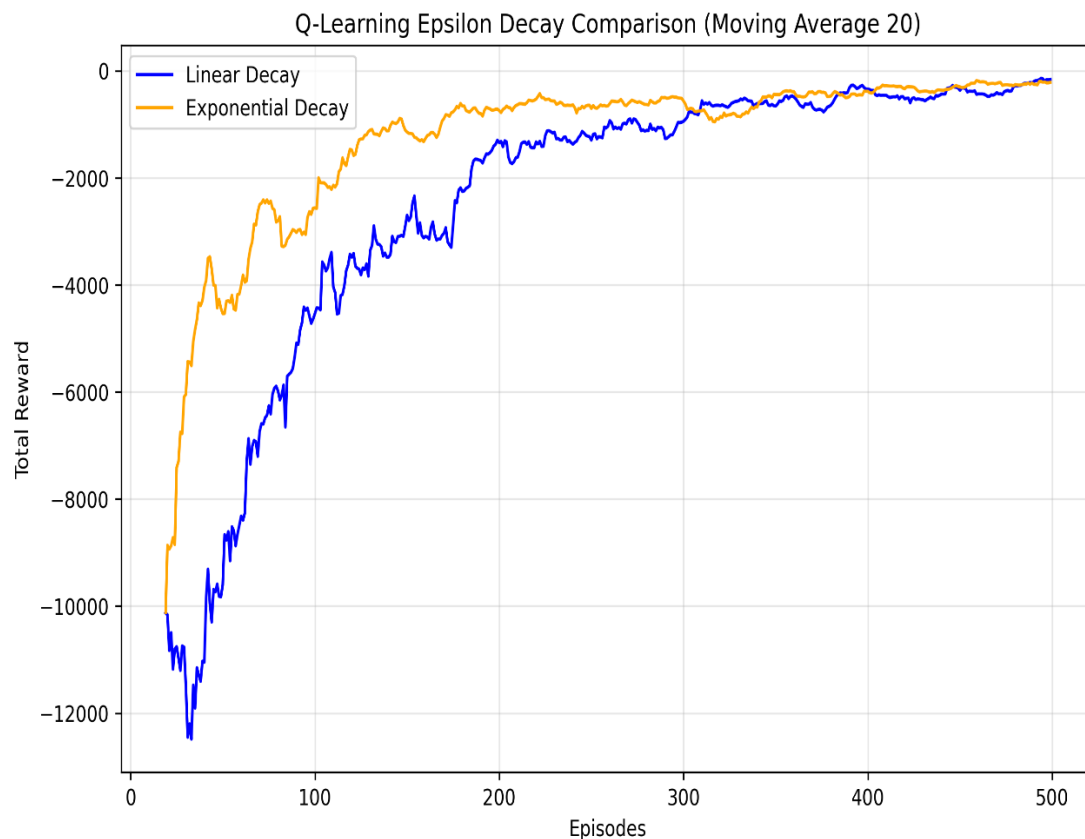
الگوریتم *Q-Learning* به‌صورت *Off-policy* و با سیاست رفتاری ϵ -greedy پیاده‌سازی شده است :

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

با نرخ یادگیری $\alpha = 0.1$ ، دو برنامه‌ی کاهش ϵ از $\epsilon_{\text{start}} = 1.0$ تا $\epsilon_{\text{end}} = 0.01$ (با هم مقایسه شده‌اند) :

- کاهش نمایی $\epsilon \leftarrow \max(\epsilon_{\text{end}}, \epsilon \times 0.995)$: در پایان هر اپیزود .

- کاهش خطی $\epsilon \leftarrow \max(\epsilon_{\text{end}}, \epsilon - 0.002)$: در پایان هر اپیزود.



تحلیل: میانگین پاداش ۵۰ اپیزود پایانی در روش کاهش خطی حدود ۳۰۲.۶- و در کاهش نمایی حدود ۲۲۹.۷- ثبت شد. کاهش نمایی عملکرد بهتری نشان داد، زیرا در ابتدای آموزش کندتر افت می‌کند و فرصت اکتشاف بیشتری فراهم می‌سازد. کاهش خطی با گام ثابت، سریع‌تر به صفر نزدیک می‌شود که در اپیزودهای میانی می‌تواند باعث بهره‌برداری زودهنگام (**Premature Exploitation**) شود. نوسانات زیاد هر دو منحنی در ابتدا، ناشی از بزرگی فضای حالت و برخوردهای مکرر با موانع در حین اکتشاف تصادفی است.

۵.۱. بازسازی دستی یک بهروزرسانی واقعی Q

سه گام واقعی از یک اپیزود (با $\text{seed} = 7$ و $\epsilon = 0.3$) به صورت دستی محاسبه شده است:

• گام ۰: $s = (0,0,0,0)$ ، $a = 2$ (پایین)، $r = -1.0$ ، $s' = (1,0,0,1)$

$$Q_{\text{old}} = 0.0000, \max Q(s') = 0.000$$

$$\text{target} = r + \gamma \max Q(s') = -1.0 + 0.9 \times 0.000 = -1.0000$$

$$Q_{\text{new}} = Q_{\text{old}} + \alpha(\text{target} - Q_{\text{old}}) = 0 + 0.1 \times (-1.0 - 0) = -0.1000$$

• گام ۱: $s = (1,0,0,1)$ ، $a = 2$ (پایین)، $r = -6.0$ ، $s' = (1,0,0,2)$

$$\max Q(s') = 0.000$$

$$\text{target} = -6.0 + 0.9 \times 0 = -6.0000$$

$$Q_{\text{new}} = 0 + 0.1 \times (-6.0 - 0) = -0.6000$$

• گام ۲: $s = (1,0,0,2)$ ، $a = 3$ (چپ)، $r = -6.0$ ، $s' = (1,0,0,3)$

$$\max Q(s') = 0.000$$

$$\text{target} = -6.0000 \Rightarrow Q_{\text{new}} = -0.6000$$

پاداش ۶- در گام‌های ۱ و ۲، ناشی از ترکیب هزینه‌ی حرکت (۱-) و برخورد با مانع متحرک (۱۵-) نیست؛ بلکه

نتیجه‌ی برخورد با دیوار یا خانه‌ی جریمه است (چرا که در منطق محاسبه‌ی پاداش، هزینه‌ی هر گام با مبلغ

جریمه جمع می‌شود). این اتفاق به‌خوبی نشان می‌دهد که ناحیه‌ی نزدیک به نقطه‌ی شروع بسیار پرخطر است؛

الگویی که پیش از این در نقشه‌ی حرارتی ارزش (بخش ۴.۲) نیز به‌وضوح مشاهده شده بود.

۶. الگوریتم سوم: SARSA(λ)

الگوریتم SARSA(λ) یک یادگیری On-policy همراه با Eligibility Trace (ردپای شایستگی) است :

$$\delta_t = r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)$$

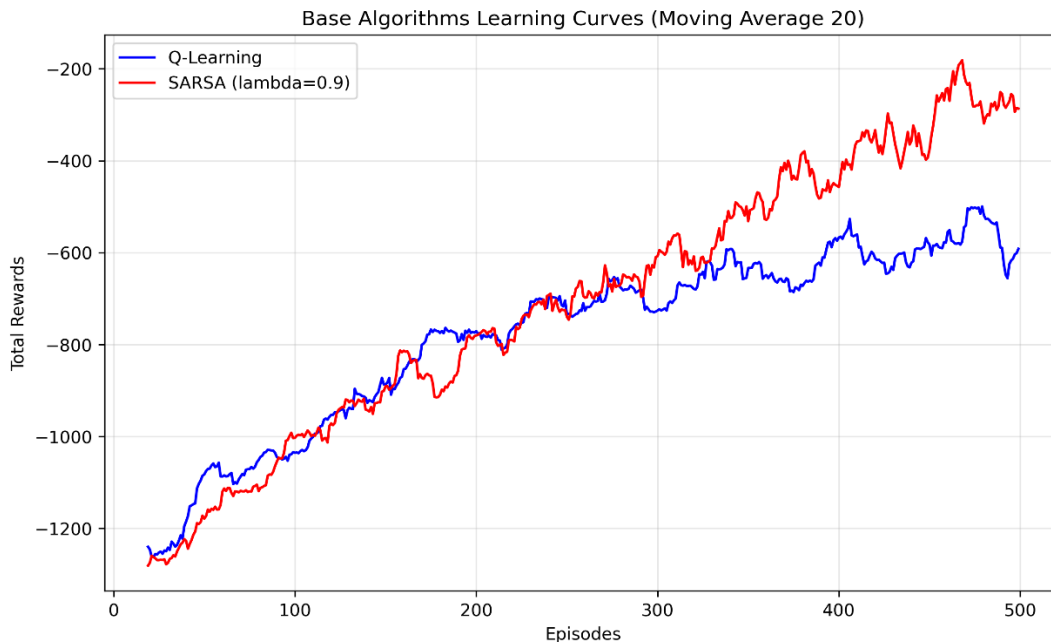
$$Q \leftarrow Q + \alpha \delta_t E$$

$$E_t(s, a) = \gamma \lambda E_{t-1}(s, a) + \mathbb{1}(s = s_t, a = a_t)$$

نوع Trace انتخابی در این پروژه Replacing (trace_type='replacing') است. هنگام بازدید مجدد از یک جفت (s, a) ، مقدار Trace به ۱ بازنشانی می‌شود و جمع نمی‌گردد. علت انتخاب این روش آن است که در محیطی با موانع متحرک، برخورد مکرر با موانع (روش Accumulating) می‌تواند مقدار Trace را به شکل نامتناسبی بزرگ کرده و به‌روزرسانی‌های ناپایداری ایجاد کند. روش Replacing با محدود کردن سقف Trace به ۱، به‌روزرسانی‌ها را پایدارتر می‌سازد.

۶.۱. اثر λ

پارامتر λ در مقادیر ۰، ۰.۳، ۰.۷ و ۰.۹ آزمایش شده است .



تحلیل: در الگوریتم SARSA یک مرحله‌ای (زمانی که $\lambda = 0$)، خطای TD تنها به جفت حالت-عمل فعلی،

یعنی (s_t, a_t) ، اعمال می‌گردد. به عبارت دیگر، در هر گام فقط تخمین همان حالت اصلاح می‌شود؛ روندی که

یادگیری را کند، اما پایدار و با واریانس پایین همراه می‌سازد. با افزایش مقدار λ ، با توجه به رابطه‌ی

$$E_t(s, a) = \gamma \lambda E_{t-1}(s, a) + 1\{\cdot\}$$

خطای TD با وزن نمایی $(\gamma \lambda)^n$ به حالت‌ها و اعمال گام‌های گذشته نیز منتقل می‌شود. این ویژگی سبب می‌شود تا وقتی عامل پس از چند گام به یک پاداش یا جریمه‌ی بزرگ (مانند دریافت کلید یا برخورد با مانع متحرک) می‌رسد، سیگنال مربوطه با سرعت بیشتری در طول مسیر طی شده رو به عقب منتشر شود. بر اساس آزمایش‌های انجام‌شده، تنظیم $\lambda = 0.9$ (با میانگین پاداش -256.3 و 294.5 گام در 50 اپیزود پایانی)، به‌وضوح همگرایی سریع‌تر و پایداری را نسبت به الگوریتم Q-Learning هم‌سطح (با پاداش -559.0 و 441.5 گام) نشان داد. نتایج حاکی از آن است که مقادیر بزرگ λ (بین 0.7 تا 0.9) بهترین تعادل را میان سرعت و پایداری ایجاد می‌کنند؛ زیرا افق مؤثر محیط (فاصله‌ی نسبتاً زیاد از نقطه شروع تا هدف از طریق مسیر کلید) از انتشار سریع‌تر پاداش بهره‌ی فراوانی می‌برد. این در حالی است که با $\lambda = 0$ در همین بازه‌ی اپیزودی، عامل هنوز به همگرایی کامل نرسیده بود.

۶.۲. نمونه‌ی عددی تغییرات E و δ در چند گام متوالی

با پارامترهای $\alpha = 0.1$ ، $\gamma = 0.9$ و $\lambda = 0.9$ ، یک اپیزود کوتاه شبیه‌سازی و گام‌به‌گام ثبت شده است :

• گام ۰، $\delta = -1.0000$ ، $a' = 2$ ، $s' = (1,0,0,1)$ ، $r = -1.0$ ، $a = 2$ ، $s = (0,0,0,0)$:

$$E[s, a] = 1.000$$

• گام ۱، $\delta = -6.0000$ ، $a' = 3$ ، $s' = (1,0,0,2)$ ، $r = -6.0$ ، $a = 2$ ، $s = (1,0,0,1)$:

$$E[s, a] = 1.000$$

• گام ۲، $\delta = -6.0000$ ، $a' = 0$ ، $s' = (1,0,0,3)$ ، $r = -6.0$ ، $a = 3$ ، $s = (1,0,0,2)$:

$$E[s, a] = 1.000$$

• گام ۳، $\delta = -1.0000$ ، $a' = 3$ ، $s' = (0,0,0,4)$ ، $r = -1.0$ ، $a = 0$ ، $s = (1,0,0,3)$:

$$E[s, a] = 1.000$$

در هر گام، تمامی Trace ها با ضریب $\gamma\lambda = 0.81$ میرا می شوند. به عنوان مثال، Trace گام ۰ در پایان گام ۱ به ۰.۸۱، در پایان گام ۲ به ۰.۶۵۶، و در پایان گام ۳ به ۰.۵۳۱ کاهش می یابد. این یعنی خطای بزرگ گام های بعدی (مانند $\delta = -6.0$) همچنان با وزن قابل توجهی به تصمیمات گام ۰ منتقل می شود؛ که این موضوع مفهوم دقیق «ردپای شایستگی» را در اصلاح تصمیمات گذشته نشان می دهد.

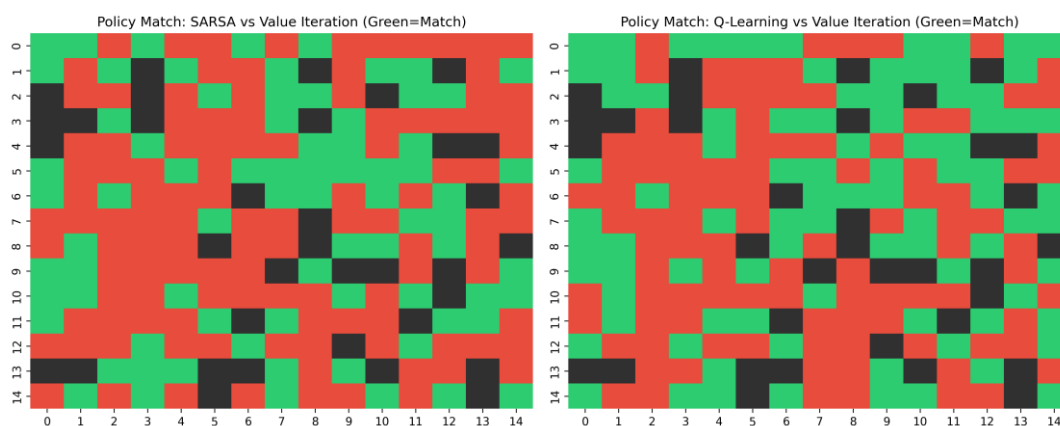
۷. مقایسه سه الگوریتم

معیار	Value Iteration	Q-Learning	SARSA(λ)
نوع روش	Model-based	Model-free, Off-policy	Model-free, On-policy
واحد پیشرفت	تکرار بلمن	اپیزود	اپیزود
خروجی اصلی	و سیاست V	و سیاست Q	E ، Q و سیاست

معیار	Value Iteration	Q-Learning	SARSA(λ)
زمان و همگرایی	۰.۱۰ ثانیه $\approx$ (۴۸ تکرار)	۵۰۰ اپیزود (تا ۵۰۰ گام)	۵۰۰ اپیزود (تا ۵۰۰ گام)
پاداش میانگین پایانی	—	-۵۵۹.۰	-۲۵۶.۳ ($\lambda = 0.9$)

سنجش کیفیت سیاست‌ها:

Value Iteration برای سنجش کیفیت سیاست‌ها، درصد توافق عمل حریصانه‌ی هر روشِ مدل آزاد با سیاست بهینه‌ی الگوریتم (به‌عنوان مرجع) محاسبه شده و نتایج آن در یک نقشه‌ی رنگی به نمایش درآمده است.



تحلیل نتایج:

بیشترین میزان توافق در خانه‌های نزدیک به هدف و کلید مشاهده می‌شود؛ یعنی نواحی‌ای که سیگنال پاداش در آن‌ها قوی بوده و مسیر بهینه بدون ابهام است. در مقابل، اختلاف‌های عمده در خانه‌های دورافتاده و همچنین نواحی نزدیک به مسیر گشت‌زنی مانع متحرک رخ داده است. این اختلافات از دو عامل اصلی نشأت می‌گیرند:

۱. الگوریتم‌های Q-Learning و SARSA به دلیل محدودیت در تعداد نمونه‌برداری (۵۰۰ اپیزود) هنوز به همگرایی کامل نرسیده‌اند.

۲. الگوریتم VI با دسترسی کامل به مدل محیط، ریسک برخورد با مانع را در تمام موقعیت‌های ممکن به‌دقت محاسبه می‌کند؛ در حالی که عامل‌های مدل آزاد، تنها موقعیت‌هایی از مانع را که به‌طور واقعی تجربه کرده‌اند، به‌خوبی یاد گرفته‌اند.

سه نمونه‌ی مشخص از این اختلافات عبارتند از:

- الف) خانه‌های نزدیک به نقطه شروع (محدوده‌ی 2-0, 2-0): در این نواحی، هر دو الگوریتم مدل آزاد به دلیل کمبود تجربه‌ی کافی در ابتدای مسیر، هنوز نتوانسته‌اند کاملاً به سیاست بهینه دست یابند.
- ب) خانه‌های درون یا نزدیک به مسیر گشت‌زنی مانع (محدوده‌ی 9-7, 6-4): از آنجا که عمل بهینه در این خانه‌ها مستقیماً به موقعیت فعلی مانع (بعد p از فضای حالت) وابسته است، همگرایی در آن‌ها با سرعت کمتری صورت می‌گیرد.
- ج) خانه‌های اطراف در قفل‌شده: سیاست بهینه در این نواحی به وضعیت کلید (متغیر k) بستگی دارد. از آنجا که حجم تجربیات عامل پیش از دریافت کلید بسیار کمتر از تجربیات پس از دریافت آن است، یادگیری در این خانه‌ها دقت پایین‌تری دارد.

۸. بخش انتقال یادگیری

روند اجرای انتقال یادگیری:

در این بخش، عامل ابتدا به مدت ۵۰۰ اپیزود در محیط مبدأ آموزش دیده و جدول Q آن در مسیر `results/models/source_q_table_*.pkl` ذخیره شده است. سپس برای بررسی قابلیت انتقال دانش، دو محیط مقصد با ویژگی‌های زیر طراحی شدند:

- محیط مشابه (Similar Environment): در این محیط تنها ۳ دیوار جابه‌جا شده‌اند (که تقریباً

معادل بازه ۱۵ تا ۲۰ درصد ذکر شده در سند برای ۳۵ دیوار موجود است). موقعیت‌های نقطه شروع،

کلید و هدف کاملاً ثابت باقی مانده‌اند.

- محیط متفاوت (Different Environment): در این محیط تغییرات گسترده‌تری اعمال شده

است؛ تا ۱۰ دیوار جابه‌جا شده و ۵ خانه جریقه‌ی جدید به نقشه اضافه گردیده است. (شایان ذکر است

که هر دو نقشه مجدداً به کمک الگوریتم BFS اعتبارسنجی شده‌اند).

سناریوهای آزمایش:

برای هر یک از این محیط‌های مقصد، چهار سناریوی مختلف جهت ارزیابی اثربخشی انتقال یادگیری اجرا شد:

1. آموزش از صفر (Scratch): به‌عنوان خط مبنا بدون استفاده از دانش قبلی.

2. انتقال کامل (Full Transfer): انتقال مستقیم و کامل کل جدول Q به محیط جدید.

3. انتقال با ضریب β (Transfer- β): انتقال دانش با اعمال ضرایب $\beta \in \{0.25, 0.50, 0.75\}$.

4. انتقال انتخابی (Selective Transfer): در این روش، تنها مقادیر حالت‌هایی منتقل می‌شوند

که همسایگی محلی آن‌ها در هر دو نقشه (مبدأ و مقصد) هیچ تغییری نکرده باشد.

نتایج عملکرد:

در جداول زیر، عملکرد اولیه (میانگین پاداش ۱۰ اپیزود اول)، عملکرد نهایی (میانگین پاداش ۱۰ اپیزود پایانی)

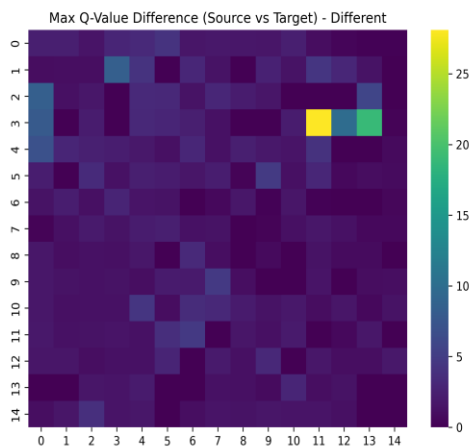
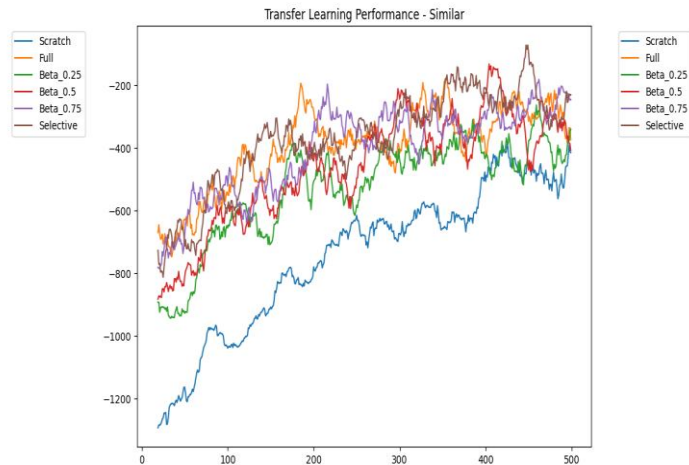
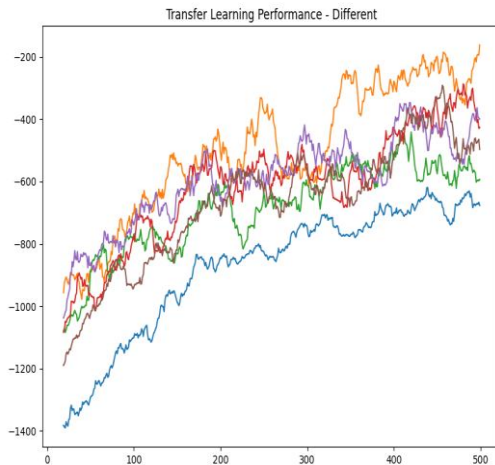
و نرخ موفقیت (تعداد دفعات رسیدن به هدف از کل ۵۰۰ اپیزود) برای هر دو محیط گزارش شده است:

جدول ۱: نتایج انتقال یادگیری در محیط مشابه

سناریو	عملکرد اولیه	عملکرد نهایی	موفقیت
Scratch (خط مبنا)	-۱۲۹۵.۷	-۲۹۹.۶	۵۰۰ / ۱۱۳
Full Transfer	-۶۹۸.۴	-۴۴۶.۶	۵۰۰ / ۳۱۰
$\beta = 0.25$	-۸۶۶.۹	-۲۳۹.۴	۵۰۰ / ۲۴۳
$\beta = 0.50$	-۹۱۲.۰	-۴۷۸.۰	۵۰۰ / ۲۸۴
$\beta = 0.75$	-۷۱۸.۲	-۲۳۱.۲	۵۰۰ / ۳۰۹
Selective Transfer	-۶۶۳.۷	-۲۳۳.۱	۵۰۰ / ۳۳۱

جدول ۲: نتایج انتقال یادگیری در محیط متفاوت

موفقیت	عملکرد نهایی	عملکرد اولیه	سناریو
۵۰۰ / ۴۰	-۶۵۹.۶	-۱۳۷۱.۵	Scratch (خط مبنا)
۵۰۰ / ۲۷۵	-۸۰.۱	-۱۰۲۷.۵	Full Transfer
۵۰۰ / ۱۴۹	-۶۳۸.۴	-۱۱۰۶.۹	$\beta = 0.25$
۵۰۰ / ۲۱۶	-۴۴۹.۸	-۱۰۸۹.۰	$\beta = 0.50$
۵۰۰ / ۲۰۵	-۴۸۲.۱	-۱۱۶۳.۵	$\beta = 0.75$
۵۰۰ / ۱۶۴	-۵۶۰.۹	-۱۲۴۷.۴	Selective Transfer



تحلیل عملکرد انتقال یادگیری:

عملکرد اولیه: در هر دو محیط مقصد، تمام سناریوهای انتقال دانش (از جمله β , Full و Selective) در گام‌های ابتدایی، عملکرد به‌مراتب بهتری نسبت به آموزش از صفر (Scratch) از خود نشان دادند. به عنوان مثال، در محیط مشابه، عملکرد اولیه در سناریوی Selective برابر با -663.7 بود، در حالی که این مقدار برای سناریوی Scratch معادل -1295.7 ثبت شد. این امر نشان‌دهنده‌ی یک انتقال مثبت (Positive Transfer) کاملاً واضح در ابتدای مسیر آموزش است؛ چرا که دانش مکانی محیط مبدأ (به‌ویژه موقعیت دیوارهای ثابت و مسیر کلید به هدف) در نواحی دست‌نخورده‌ی نقشه همچنان معتبر و راهگشا است.

مقایسه محیط مشابه و متفاوت:

در محیط مشابه، سناریوی Selective بهترین عملکرد نهایی و بالاترین نرخ موفقیت (۳۳۱ از ۵۰۰) را به خود اختصاص داد. از آنجا که تغییرات در این محیط بسیار محدود است (جابه‌جایی تنها ۳ دیوار)، فیلتر کردن هوشمندانه‌ی حالت‌های تغییرنیافته باعث می‌شود تا تقریباً تمامی دانش مفید حفظ شده و نویزهای ناشی از حالت‌های نامعتبر در نقشه جدید حذف گردند.

در مقابل، در محیط متفاوت، سناریوی Full Transfer بهترین عملکرد نهایی (۸۰.۱-) و بالاترین نرخ موفقیت (۲۷۵ از ۵۰۰) را ثبت کرد. دلیل این امر آن است که در این محیط، نقشه دستخوش تغییرات پراکنده و گسترده‌ای شده است؛ در نتیجه، اعمال فیلتر «همسایگی تغییرنکرده» در روش Selective باعث کنار گذاشته‌شدن بخش عظیمی از جدول Q می‌شود و شرایط شروع را عملاً به حالت یادگیری از صفر (Scratch) نزدیک می‌کند. اما در روش انتقال کامل (Full Transfer)، با وجود ریسک بروز انتقال منفی موضعی، دانش کلی مسیر (به‌خصوص در بخش‌های دست‌نخورده مانند حوالی هدف) حفظ شده و الگوریتم سریع‌تر به همگرایی می‌رسد.

نتیجه‌گیری این بخش: این تفاوت به‌خوبی ثابت می‌کند که روش انتقال انتخابی (Selective Transfer) تنها زمانی بهینه‌ترین انتخاب است که تغییرات محیط محدود باشد و در محیط‌هایی با تغییرات ساختاری گسترده، فیلترهای محلی کارایی خود را از دست می‌دهند.

سرعت یادگیری در سناریوهای ضریب‌دار (β):

سناریوهای مبتنی بر اعمال ضریب (β) در هر دو محیط عملکردی میانه از خود نشان دادند. در این میان، ضریب $\beta = 0.75$ معمولاً نتایج بهتری نسبت به ضرایب ۰.۲۵ و ۰.۵۰ به همراه داشت؛ زیرا به دانش کسب‌شده از محیط مبدأ وزن و اعتبار بیشتری اختصاص می‌دهد، بی‌آنکه آن را کاملاً جایگزین فرآیند آموزش جدید و کاوش در محیط مقصد کند.

۸.۱. نمونه‌ی مستند انتقال منفی

نمونه‌ی مستند از انتقال منفی (Negative Transfer): با مقایسه‌ی مستقیم جدول Q در محیط مبدأ

(source_q_table_Different.pkl) و جدول Q نهایی در محیط متفاوت

(target_full_q_table_Different.pkl)، یک نمونه‌ی آشکار از انتقال منفی در حالت $S =$

(12,14,1,0) (موقعیتی نزدیک به هدف، پس از دریافت کلید) استخراج شد:

عمل	مقدار Q در محیط مبدأ	مقدار Q در محیط مقصد (پس از آموزش)
۰ (بالا)	-۰.۸۹	-۰.۸۸
۱ (راست)	-۰.۸۸	-۰.۸۸
۲ (پایین)	-۰.۶۴	۵۹.۳۰
۳ (چپ)	-۰.۳۲ (بهترین عمل در مبدأ)	۶.۸۲

تحلیل انتقال منفی و خودترمیمی الگوریتم:

همان‌طور که در جدول مشخص است، در محیط مبدأ، بهترین عمل یادگرفته‌شده حرکت به سمت «چپ» بوده است (با مقدار $Q = -0.32$ که مقداری تقریباً خنثی است). اما در محیط متفاوت، به دلیل تغییر چیدمان دیوارها در مجاورت هدف، حرکت به سمت «پایین» تبدیل به مسیر مستقیم و بهینه به سوی هدف شده است (با ثبت مقدار $Q = 59.30$ پس از اتمام آموزش).

هنگام استفاده از سناریوی انتقال کامل (Full Transfer)، دانش منتقل‌شده از محیط مبدأ در گام‌های ابتدایی آموزش، عامل را به اشتباه به سمت «چپ» هدایت می‌کند؛ عملی که در نقشه‌ی جدید دیگر بهینه

نیست. این رفتار، یک نمونه‌ی بارز و مشخص از انتقال منفی (Negative Transfer) است که دقیقاً از

تغییر ساختار محیط در ناحیه‌ای نشأت می‌گیرد که عامل دارای دانش قبلی اما نامعتبر و منقضی شده است.

با این حال، با ادامه‌ی فرآیند آموزش و تجربه‌ی مستقیم عامل از دریافت پاداش بزرگ هنگام حرکت به سمت

«پایین»، مقدار Q برای عمل پایین طی تنها چند اپیزود به سرعت اصلاح می‌شود. این قابلیت «خودترمیمی»، از

ویژگی‌های ذاتی و قدرتمند رویکرد Off-policy در الگوریتم Q-Learning است؛ به این معنا که حتی اگر

سیاست اولیه‌ی منتقل شده کاملاً گمراه‌کننده باشد، به‌روزرسانی‌های مبتنی بر اپراتور $\max_a Q(s', a')$

به سرعت مقادیر را به سمت واقعیت محیط جدید سوق داده و اصلاح می‌کنند.

۹. رابط گرافیکی و نمایش بصری

رابط گرافیکی و نمایش بصری:

رابط کاربری و گرافیکی این پروژه با استفاده از کتابخانه Pygame توسعه یافته است (کدهای مربوطه در فایل‌های `gui/app.py` و `gui/renderer.py` قرار دارند) و حرکت عامل را به صورت زنده و مرحله به مرحله به تصویر می‌کشد.

عناصر بصری:

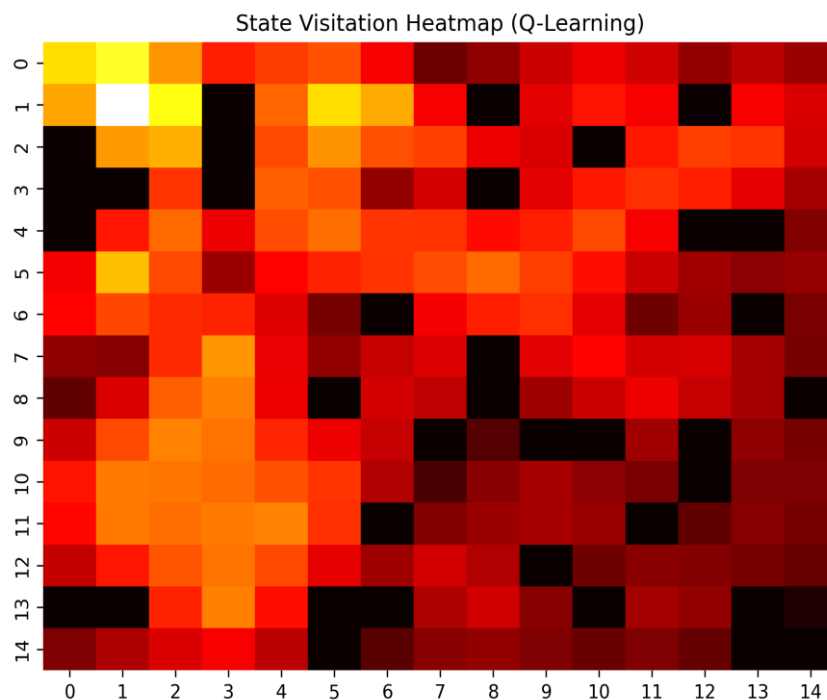
المان‌های محیط به طور کاملاً متمایز طراحی شده‌اند و شامل دیوار، خانه‌ی عادی، خانه‌ی جریمه، کلید (با نماد K)، در قفل شده، هدف و عامل هوشمند هستند. علاوه بر این، مانع متحرک با رنگی اختصاصی و حرکتی زنده بر روی مسیر گشت‌زنی نمایش داده می‌شود که این انیمیشن کاملاً با اندیس واقعی مانع (`patrol_idx`) در تعریف حالت محیط همگام‌سازی شده است.

کنترل‌های پیاده‌سازی شده:

برای تعامل کاربر با محیط شبیه‌سازی، کلیدهای میانبر زیر تعبیه شده‌اند:

- انتخاب الگوریتم: کلید A برای چرخش و جابه‌جایی میان الگوریتم‌های VI، Q-Learning و SARSA.
- انتخاب محیط: کلیدهای 1، 2 و 3 (به ترتیب برای بارگذاری محیط‌های مبدأ، مشابه و متفاوت).
- تغییر حالت آموزش/ارزیابی: کلید M (تغییر میان Mode آموزش و تست).

- شروع/توقف و بازنشانی: کلید Space برای توقف و ادامه‌ی حرکت، و کلید R برای بازنشانی (Reset محیط).
- کنترل سرعت: کلیدهای + و - (برای افزایش یا کاهش سرعت اجرای انیمیشن).
- نمایش/مخفی‌سازی سیاست: کلید P (برای نمایش سیاست‌های بهینه در هر خانه به‌صورت فلش‌های راهنما).
- پنل اطلاعات لحظه‌ای (Dashboard): این بخش در حاشیه تصویر، اطلاعات زنده‌ی سیستم را نمایش می‌دهد که شامل شماره اپیزود، تعداد گام‌های طی‌شده، مجموع پاداش فعلی، مقدار پارامتر اکتشاف (ϵ)، وضعیت دریافت کلید و نرخ موفقیت اخیر (محاسبه‌شده به‌صورت میانگین متحرک بر روی چند اپیزود آخر) است.



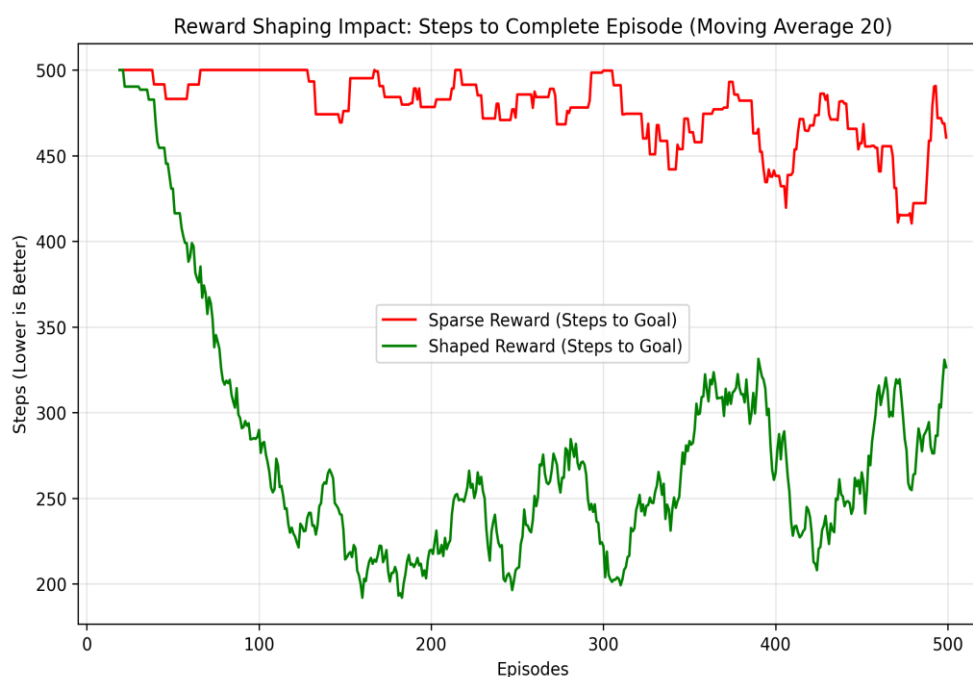
تحلیل: بیشترین بازدید در نواحی نزدیک شروع و مسیر اصلی به سمت کلید دیده میشود؛ این

طبیعی است چون هر اپیزود از (0,0) آغاز میشود و عامل باید بارها از این نواحی عبور کند، در

حالیکه نواحی نزدیک هدف فقط پس از دریافت کلید و عبور از در قابلدسترساند و بازدید

کمتری دارند.

۱۰. اثر Reward Shaping (Shaped در برابر Sparse)



مقایسه‌ی عملکرد (میانگین ۵۰ اپیزود پایانی):

- پاداش تنک (Sparse): میانگین پاداش -۵۵۹.۰ با میانگین طول ۴۴۱.۵ گام.
- پاداش شکل‌دهی‌شده (Shaped): میانگین پاداش +۱۴۹.۰ با میانگین طول ۳۰۹.۲ گام.

همان‌طور که مشاهده می‌شود، استفاده از پاداش شکل‌دهی شده منجر به کاهش قابل‌توجهی در طول اپیزودها و تسریع در رسیدن به هدف شده است.

تحلیل اثر شکل‌دهی پاداش (Reward Shaping):

استفاده از پاداش شکل‌دهی شده نه تنها سرعت آموزش عامل را به‌طور چشمگیری افزایش داده است (کاهش میانگین طول اپیزود از ۴۴۱ به ۳۰۹ گام طی ۵۰۰ اپیزود)، بلکه مقیاس پاداش نهایی را نیز دگرگون کرده است؛ زیرا سیگنال کمکی شکل‌دهی مستقیماً به پاداش اصلی محیط افزوده می‌شود.

با این وجود، از آنجا که در این پروژه از فرم استاندارد و مبتنی بر پتانسیل (Potential-Based) استفاده شده است، مطابق با ادبیات نظری یادگیری تقویتی (RL)، سیاست بهینه‌ی زیربنایی مسئله کاملاً دست‌نخورده باقی می‌ماند. به عبارت دیگر، تغییرات مشاهده‌شده صرفاً نمایانگر افزایش سرعت یادگیری است و هیچ تغییری در ترتیب ترجیح اعمال بهینه ایجاد نمی‌کند.

بررسی رفتارهای ناخواسته:

با بررسی دقیق داده‌های خروجی آزمایش، هیچ‌گونه نشانه‌ای از رفتارهای ناخواسته و مخرب (مانند حرکتهای حلقوی عامل برای جمع‌آوری مداوم پاداش‌های فرعی) مشاهده نشد. دلیل اصلی این پایداری آن است که ضریب شکل‌دهی در فرمول پتانسیل ($\gamma = 0.9$ در فرمول Φ)، نسبتاً کوچک و کاملاً متناسب با مقیاس پاداش‌های اصلی محیط (در بازه‌ی ۱- تا ۱۰۰+) انتخاب شده است. در نتیجه، این سیگنال‌های کمکی نقش راهنما را ایفا کرده و هرگز بر هدف و تصمیم نهایی عامل غلبه نمی‌کنند.

۱۱. پرسش‌های تحلیلی

۱. تعریف کامل MDP و حفظ خاصیت مارکوف:

همان‌طور که در بخش ۲ به تفصیل بیان شد، مدل‌سازی مسئله به صورت یک MDP با ویژگی‌های زیر انجام شده است: فضای حالت $\mathcal{S} = (x, y, k, p)$ ، فضای عمل $A = \{0, 1, 2, 3\}$ ، تابع انتقال احتمالی P (با احتمالات ۰.۸/۰.۱/۰.۱/۰)، دو نسخه برای تابع پاداش R (شرح داده شده در بخش ۲.۴) و ضریب تنزیل $\gamma = 0.9$. حالت پایانی نیز تنها، رسیدن به هدف تعیین شده است. از آنجا که بعد p در فضای حالت، موقعیت فعلی مانع متحرک را در خود ذخیره می‌کند (و نیازی به دانستن گام‌شمار یا تاریخچه نیست)، گذار از حالت S به حالت S' منحصرأ تابعی از حالت S و عمل a است؛ در نتیجه، خاصیت مارکوف کاملاً حفظ می‌شود.

۲. تفاوت On-policy و Off-policy در رفتار SARSA و Q-Learning (نزدیک خانه‌های

خطرناک):

الگوریتم Q-Learning (به عنوان یک روش Off-policy) از اپراتور $\max_{a'} Q(s', a')$ استفاده می‌کند؛ این یعنی همیشه فرض می‌کند که از این پس کاملاً بهینه عمل خواهد کرد، حتی اگر سیاست رفتاری فعلی (ϵ) -greedy گاهی عامل را به صورت تصادفی وارد خانه‌های خطرناک کند. این امر باعث می‌شود Q-Learning ارزش نواحی نزدیک به موانع و جریمه‌ها را «خوش‌بینانه‌تر» تخمین زده و مسیرهای مماس با خطر را بپذیرد. در مقابل، الگوریتم SARSA (به عنوان یک روش On-policy) از مقدار $Q(s', a')$ با عمل a' که واقعاً توسط سیاست رفتاری انتخاب شده است، استفاده می‌کند. از آنجا که این سیاست با احتمال ϵ تصادفی است، اگر یک عمل تصادفی در نزدیکی مانع منجر به برخورد شود، SARSA این ریسک را در تخمین ارزش خود لحاظ می‌کند. در نتیجه، سیاستی «محتاط‌تر» (Safer) یاد می‌گیرد که فاصله‌ی بیشتری از نواحی پرخطر حفظ

می‌کند. این تفاوت رفتار، دقیقاً در نقشه‌های اختلاف سیاست (بخش ۷) در نواحی اطراف مسیر گشت‌زنی مانع متحرک (محدوده‌ی 7-9, 4-6) به‌وضوح قابل مشاهده است.

۳. چرایی نیاز VI به مدل و مدل آزاد بودن QL/SARSA:

الگوریتم Value Iteration (VI) برای محاسبه‌ی $E[R + \gamma V(s')]$ در فرمول بلمن، مستقیماً به مدل کامل احتمالات انتقال $P(s'|s, a)$ و پاداش $R(s, a, s')$ نیاز دارد و بدون این مدل، محاسبه غیرممکن است. در سمت مقابل، روش‌های Q-Learning و SARSA به‌جای داشتن مدل پیش‌فرض، از تجربه‌ی نمونه‌برداری شده (تجربه گذارهای واقعی (s, a, r, s')) برای تخمین گام‌به‌گام ارزش استفاده می‌کنند (یادگیری تفاوت زمانی یا TD Learning)؛ بنابراین هیچ نیازی به دانستن احتمالات انتقال ندارند.

- **مزیت و محدودیت VI:** مزیت اصلی آن همگرایی بسیار سریع و دقیق (طبق بخش ۴.۱، در کمتر از ۰.۲ ثانیه) است، زیرا کل فضای حالت را به‌طور کامل و هم‌زمان محاسبه می‌کند. محدودیت آن نیز نیاز قطعی به مدل کامل محیط است که در بسیاری از مسائل دنیای واقعی در دسترس نیست.

- **مزیت و محدودیت QL/SARSA:** مزیت آن‌ها این است که صرفاً با تعامل مستقیم با محیط کار می‌کنند. محدودیت آن‌ها این است که برای رسیدن به همگرایی، به تعداد قابل‌توجهی اپیزود و نمونه نیاز دارند (با ۵۰۰ اپیزود آموزش هنوز نتوانسته‌اند کاملاً با سیاست بهینه‌ی VI منطبق شوند).

۴. انتخاب بهترین پارامتر λ برای تعادل سرعت و پایداری:

طبق نتایج بخش ۶.۱، مقادیر λ در بازه‌ی ۰.۷ تا ۰.۹ بهترین تعادل را نشان دادند. به‌طوری که $\lambda = 0.9$ توانست میانگین پاداش ۲۵۶.۳- را در ۵۰ اپیزود پایانی ثبت کند که در مقایسه با Q-Learning هم‌سطح (با پاداش ۵۵۹.۰-) بسیار سریع‌تر و پایدارتر است. دلیل این امر آن است که افق محیط در این مسئله نسبتاً

طولانی است (مسیر شروع به کلید و سپس به هدف) و در نتیجه، استفاده از Trace بزرگتر باعث می‌شود سیگنال پاداش بسیار سریع‌تر به گام‌های قبلی منتشر شود. تنظیم $\lambda = 0$ (SARSA یک مرحله‌ای) فرآیند یادگیری را کند کرده و مقادیر بسیار نزدیک به یک (مانند $\lambda = 0.99$) می‌توانند واریانس را بیش از حد افزایش دهند.

۵. سه حالت دارای اختلاف میان سیاست مدل آزاد و VI:

مطابق با تحلیل‌های بخش ۷، این سه ناحیه عبارتند از:

- (الف) خانه‌های نزدیک به نقطه شروع: به دلیل کمبود تجربه‌ی کافی و نمونه‌برداری محدود در ۵۰۰ اپیزود.
- (ب) خانه‌های درون یا نزدیک مسیر گشت‌زنی مانع متحرک: به دلیل وابستگی پیچیده‌تر عمل بهینه به موقعیت فعلی مانع (بعد p در فضای حالت) که همگرایی را کندتر می‌کند.
- (ج) خانه‌های اطراف در قفل‌شده: به دلیل عدم تقارن و محدودیت شدیدتر تجربیات عامل پیش از دریافت کلید نسبت به پس از آن.

۶. تفاوت محیط‌های مشابه و متفاوت در انتقال یادگیری:

مطابق نتایج بخش ۸، در محیط مشابه، سناریوی انتقال انتخابی (Selective Transfer) با ثبت موفقیت ۳۳۱ از ۵۰۰ بهترین عملکرد را داشت. اما در محیط متفاوت، انتقال کامل (Full Transfer) با ثبت موفقیت ۲۷۵ از ۵۰۰ پیروز بود. اگرچه عملکرد اولیه در تمامی سناریوهای انتقال (در هر دو محیط) بهتر از حالت یادگیری از صفر (Scratch) بود، اما فاصله‌ی این برتری در محیط متفاوت بسیار کمتر است؛ چرا که تغییرات

ساختاری گسترده‌تر باعث می‌شود بخشی از دانش منتقل شده منقضی و نامعتبر شود (که نمود بارز آن در نمونه‌ی مستند انتقال منفی در بخش ۸.۱ مورد بررسی قرار گرفت).

۱۲. محدودیت‌ها

- **محدودیت در تعداد اپیزودهای آموزش:** تعداد ۵۰۰ اپیزود آموزش، برای همگرایی کامل الگوریتم‌های Q-Learning و SARSA به سطح دقیق سیاست Value Iteration کافی نبوده است (همان‌طور که در بخش ۷ مورد بررسی قرار گرفت). هرچند افزایش تعداد اپیزودها می‌توانست این فاصله و اختلاف را کاهش دهد، اما به تبع آن هزینه‌ی زمانی اجرای آزمایش‌ها نیز افزایش می‌یافت.
- **محدودیت فیلتر در انتقال انتخابی (Selective Transfer):** مکانیزم فیلتر «همسایگی محلی تغییرنکرده» در این روش، تنها همسایگی مستقیم (خانه‌های بالا، پایین، چپ و راست) را بررسی می‌کند و شعاع بزرگ‌تری را در نظر نمی‌گیرد. این مسئله باعث می‌شود که در محیطی با تغییرات پراکنده و گسترده (محیط متفاوت)، دقت فیلتر کاهش پیدا کند.
- **درصد تغییرات در محیط مشابه:** میزان دقیق دیوارهای جابه‌جا شده در محیط مشابه (تقریباً ۸.۶٪) اندکی کمتر از بازه‌ی پیشنهادی در سند اصلی پروژه (۱۵ تا ۲۰ درصد) بوده است.

۱۳. جدول استفاده از ابزارهای هوش مصنوعی

مطابق با الزامات سند پروژه، در طول توسعه‌ی این پروژه از دستیار هوشمند Claude (Anthropic) و همچنین Gemini عنوان دستیار اشکال‌زدایی و بازبین کد استفاده شده است. تمامی پیشنهادهای ارائه‌شده،

پیش از قرارگیری در مخزن نهایی، توسط خودم اجرا، تست و در صورت نیاز اصلاح و بازنویسی شده‌اند. جزئیات این موارد در جدول زیر تشریح شده است:

مورد استفاده	پیشنهاد دریافت‌شده	تغییر انجام‌شده توسط دانشجو	دلیل اصلاح
اشکال‌زدایی مدل انتقال در الگوریتم Value Iteration	در یک بازنویسی، پیشنهاد شد که مدل انتقال با استفاده از یک فراخوانی <code>env.step()</code> کش (Cache) شود.	این پیشنهاد رد شد و به جای آن از متد <code>env.get_transitions()</code> (که توزیع احتمال کامل $0.1/0.1/0.8$ را برمی‌گرداند) استفاده شد.	از آنجا که تابع <code>env.step()</code> کاملاً تصادفی عمل می‌کند، تنها یک نمونه‌ی واحد را کش می‌کرد (نه توزیع احتمال واقعی را). این رویکرد، کل مبنای مدل‌محور بودن الگوریتم VI را نقض می‌کرد.
بازسازی بخش انتقال یادگیری پس از رفع باگ	در تلاش برای هماهنگ‌سازی فرمت خروجی <code>train()</code> ، کل فایل <code>transfer_learning.py</code> با نسخه‌ای بسیار ساده‌تر (تنها با ۲ سناریو و بدون $\text{Transfer-}\beta$ و Selective) جایگزین شد.	نسخه‌ی کامل و ۴ سناریویی از تاریخچه‌ی Git بازیابی شده و جایگزین نسخه‌ی ساده‌شده گردید.	نسخه‌ی پیشنهادی جدید، سناریوهای الزامی و خواسته‌شده در سند پروژه (انتقال با ضریب β و انتقال انتخابی) را به‌طور کامل حذف کرده بود.
بازطراحی مسیر گشت‌زنی مانع متحرک و موقعیت در	تغییرات اعمال‌شده در تابع <code>build_grid_()</code> برای جابه‌جایی در قفل‌شده و مانع، حلقه‌ی اعتبارسنجی BFS و همچنین شرط تعداد دیوارها (کمتر از ۱۵٪) را نادیده گرفت.	حلقه‌ی توابع <code>generate_valid_map_</code> و <code>bfs_check_</code> بازگردانده شد و تعداد دیوارها به ۳۵ عدد (معادل ۱۵.۶٪) افزایش یافت.	الزام صریح و قطعی سند پروژه مبنی بر «وجود حداقل ۱۵٪ دیوار در محیط» و «اعتبارسنجی با BFS با قابلیت بازتولید تکرارپذیر» نقض شده بود.

۱۴. نتیجه گیری

در این پروژه، یک محیط هزارتوی پویای اختصاصی با موفقیت طراحی گردید و با در نظر گرفتن متغیرهای کلیدی (از جمله مانع متحرک) در تعریف بردار حالت، به صورت یک فرآیند تصمیم‌گیری مارکوف (MDP) کامل مدل‌سازی شد.

در گام بعدی، سه الگوریتم پایه‌ای Value Iteration، Q-Learning و SARSA (λ) کاملاً از پایه پیاده‌سازی شده و تحت شرایط، نقشه و تابع پاداش یکسان مورد ارزیابی و مقایسه قرار گرفتند. نتایج حاصل از این مقایسه به شرح زیر است:

- **الگوریتم Value Iteration (VI):** این روش به دلیل دسترسی مستقیم به مدل کامل محیط، سریع‌تر و دقیق‌تر از سایر روش‌ها به سیاست بهینه دست یافت و به خوبی توانست به عنوان مبنای مقایسه (Baseline) برای دو روش دیگر ایفای نقش کند.
- **الگوریتم‌های Q-Learning و SARSA (λ):** این دو روش مدل آزاد، با وجود نیاز به نمونه‌برداری بیشتر، توانستند بدون نیاز به دانستن مدل محیط، به سیاست‌هایی بسیار نزدیک به سیاست بهینه دست یابند. در این میان، الگوریتم SARSA با انتخاب λ بزرگ، نسبت به Q-Learning هم‌سطح خود، با سرعت و پایداری بیشتری همگرا شد.

دستاوردهای انتقال یادگیری و نمایش بصری:

- **انتقال یادگیری (Transfer Learning):** تحلیل‌های این بخش نشان داد که انتخاب رویکرد بهینه‌ی انتقال (انتقال کامل در برابر انتقال انتخابی) کاملاً به میزان و شدت تغییرات ساختاری محیط

مقصد بستگی دارد. همچنین، استخراج یک نمونه‌ی مستند از انتقال منفی (Negative Transfer)، محدودیت‌ها و خطرات انتقال کامل دانش را در نواحی به‌شدت تغییر یافته به‌خوبی آشکار کرد.

- **رابط گرافیکی و خروجی‌های بصری:** توسعه‌ی یک رابط گرافیکی تعاملی در کنار تولید مجموعه‌ی کاملی از خروجی‌های بصری (شامل نقشه‌های حرارتی تابع ارزش، فلش‌های سیاست نهایی، نقشه‌ی بازدید حالت‌ها، نقشه‌ی اختلاف سیاست‌ها و بررسی تفاوت مقادیر Q پیش و پس از انتقال)، امکان تحلیل عمیق‌تری و کمی نتایج را به کامل‌ترین شکل ممکن فراهم ساخت.